

Eigent Health Team — Interview Study Guide

Eigent Health Team — Interview Study Guide

A complete walkthrough of the project, written so you can explain *every* choice in the room. Read end-to-end once, then re-skim by section.

One-line pitch (memorize): *A multi-agent CAMEL Workforce that turns a personal profile and optional lab work into a personalized health plan, with agentic RAG over a curated knowledge base, a knowledge graph of nutrient relationships, live cost telemetry, a human-in-the-loop approval gate, and cost-efficient follow-up refinement.*

Table of contents

1. [Elevator pitch & 4-minute demo script](#)
 2. [System architecture](#)
 3. [The five agents](#)
 4. [CAMEL primitives used](#)
 5. [Retrieval: vector + graph + web \(agentic RAG\)](#)
 6. [Lab Parser — typed document ingestion](#)
 7. [Human-in-the-loop approval gate](#)
 8. [Follow-up refinement \(the 2-stage micro-run\)](#)
 9. [Chain-of-thought transparency](#)
 10. [Live cost telemetry](#)
 11. [Product engineering stack](#)
 12. [Evals](#)
 13. [Files map \(where everything lives\)](#)
 14. [Likely interview questions, with scripts](#)
 15. [What I'd build next \(and why I didn't\)](#)
 16. [Honest limitations](#)
-

1. Elevator pitch & 4-minute demo script

Elevator pitch (~30s):

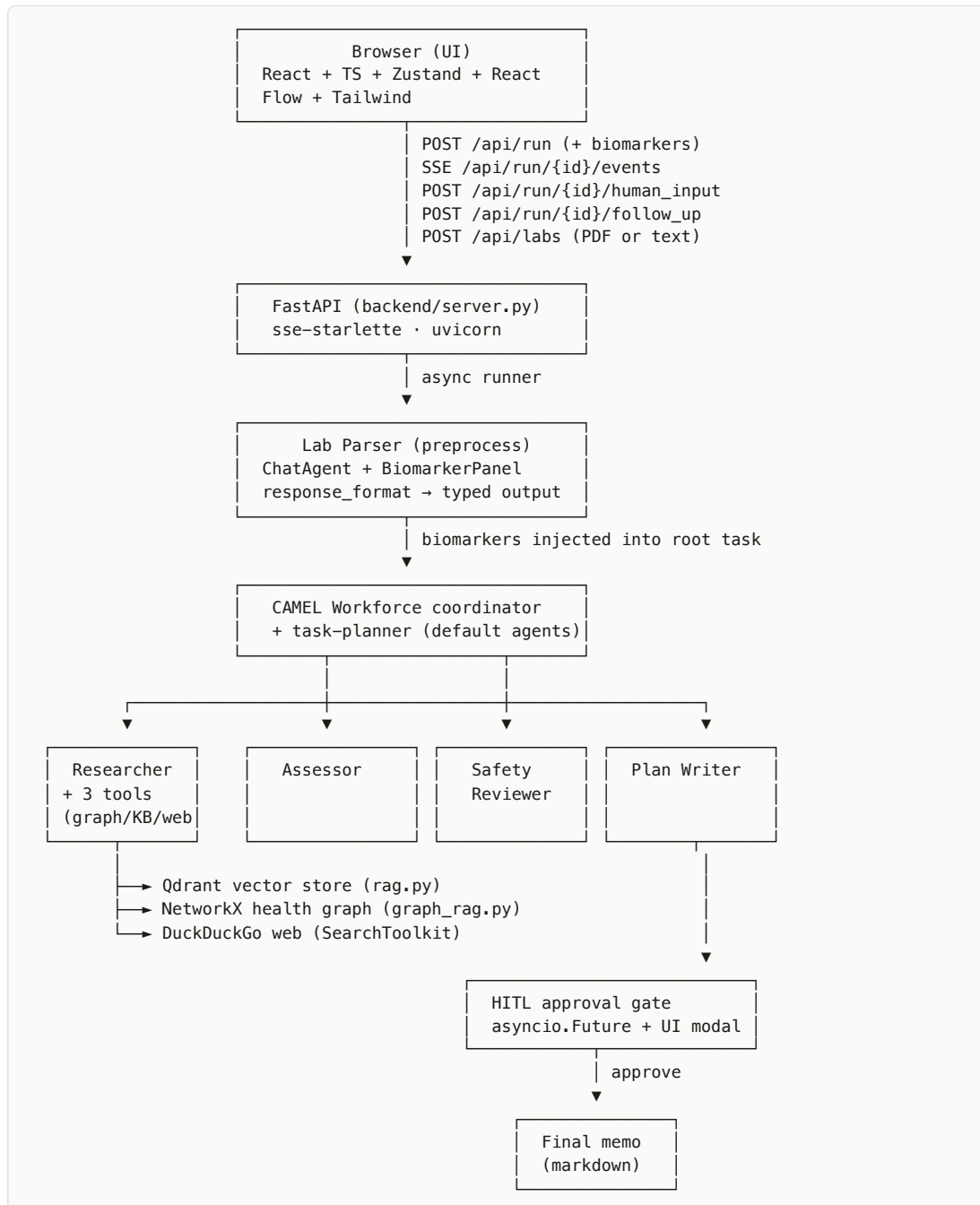
"I built a multi-agent health-planning system on the CAMEL Workforce framework — the same framework Eigent's product is built on. The user drops in a profile and optional lab work, five specialized agents collaborate: a Lab Parser extracts biomarkers, a Health Researcher consults a curated knowledge base, a knowledge graph, and the web; an Assessor picks focus areas; a Safety Reviewer pressure-tests for risks; and a Plan Writer assembles the final personalized plan. The whole run pauses for human approval before the plan is shown. The user can then follow up — 'actually my left knee hurts' — and only the Safety Reviewer and Plan Writer re-run, at a fraction of the cost."

4-minute demo script:

1. **(0:00–0:30) Setup.** Open the localhost page. Show the four worker nodes + Lab Parser node + Coordinator + Memo in React Flow.
2. **(0:30–1:00) Paste labs.** Use a stress profile: low Vitamin D, low ferritin, high LDL. Lab Parser node lights up violet → 5 biomarkers in the table.
3. **(1:00–1:30) Run.** Hit “Run” on a profile like *“42, software engineer, mostly sedentary, want better energy and sleep.”* Coordinator dispatches; workers light up amber → green as they stream.
4. **(1:30–2:30) Show tool use live.** Click the Researcher node mid-run. Drawer slides in. Show: the reasoning trace, the tool calls (graph , KB , web ) with their queries and retrieved entities/sources, and per-worker token + cost counters.
5. **(2:30–3:00) HITL gate.** When the four workers finish, the approval modal slides up: Safety Notes from the Safety Reviewer, Focus Areas, “if you only do one thing,” and the cost so far. Approve.
6. **(3:00–3:45) Plan renders.** Markdown plan with profile-grounded bullets that cite biomarker names + values; Action Plan with Today/Week/Month; “What to Avoid”; the disclaimer at the bottom.
7. **(3:45–4:00) Follow-up.** Type *“actually my left knee hurts on stairs”* in the follow-up box. Hit Refine. The graph shows Safety Reviewer + Plan Writer re-light while Researcher/Assessor stay done. ~\$0.005 added cost. New HITL → Approve → updated plan with knee-specific guidance.

Closing line: *“Five agents, three retrieval modes, live cost visibility, human-in-the-loop, cost-efficient follow-ups — same shape as Eigent’s own product, but for personalized health.”*

2. System architecture



Key data flow points to be able to explain:

- The frontend is **same-origin** with the backend in production (FastAPI serves the built React app from `frontend/dist/`); in dev, Vite proxies `/api` to `localhost:8000`.
- The runner sends **typed SSE step events** — every coordinator/worker state change emits a `RunEvent` Pydantic model with discriminator `type`.
- Streaming is **per token via worker_chunk events** thanks to OpenAI's `stream_options.include_usage` (which also unlocks live cost numbers).
- Tool calls emit **their own tool_call events** with the query + the retrieved sources or entities for the drawer.

3. The five agents

All five are **CAMEL ChatAgent s**, each defined by a `system_message`, a `model` (`ModelFactory.create(...)`), and optional `tools`. Their behavior is the prompt + the LLM — there is no procedural Python logic inside any agent. Source: `src/agents.py` and `src/lab_parser.py`.

Lab Parser (`src/lab_parser.py`)

- **Role:** standalone preprocessing step. Given raw lab text (from PDF via pypdf, or pasted), extract a typed `BiomarkerPanel`.
- **Why standalone, not a Workforce worker:** parsing is a single-shot, deterministic task — not a collaborative one. Keeping it out of the Workforce avoids the coordinator’s non-determinism and lets it run *before* the main pipeline.
- **Schema-typed output:** uses CAMEL’s `response_format=BiomarkerPanel` in `parse_labs()`. The Pydantic class is in `src/schema.py`. This is what makes downstream agents trustworthy — they’re not parsing text, they receive `[{name, value, unit, reference_range, flag}, ...]`.
- **Talking point:** “*I split parsing from reasoning. The parser is a one-shot typed extraction; the Workforce members are reasoners that see the structured biomarkers in their root-task context.*”

Health Researcher (`health_researcher_agent`)

- **Role:** gather evidence-grounded information.
- **Tools (3):**
 - `query_health_graph(query)` — NetworkX-backed relational retrieval.
 - `search_health_kb(query)` — Qdrant vector retrieval over curated docs.
 - `search_duckduckgo(query)` — open web fallback.
- **Routing is in the prompt** — “*prefer the graph for relational questions, the KB for guidelines, the web for fresh/specific things.*” This is **agentic** in the literal sense: the LLM, not the runtime, decides which tool to call.
- **In practice:** on a typical health profile, ~2 graph calls + 1–2 KB calls + 0 web calls. The Researcher cites source URLs and entity names.

Health Assessor (`health_assessor_agent`)

- **Role:** read the profile + research, pick 3–4 high-impact focus areas with measurable baselines + targets + a “first concrete step today.”
- **No tools.** Pure reasoning.
- **Key prompt rule:** every focus area must have a **number** — baseline, target, frequency. Forces concreteness.

Safety Reviewer (`safety_reviewer_agent`)

- **Role:** pressure-test the emerging plan for risks, contraindications, red flags.
- **Schema-typed (designed for):** `src/schema.py` defines `SafetyReview` with `risks`, `consult_a_professional`, `verdict` ∈ {`safe-to-follow`, `follow-with-caution`, `consult-first`}, `one_line_summary`. Inside the Workforce, the agent emits markdown matching this shape; the `evals/deterministic.py` test calls the same agent directly with `response_format=SafetyReview` and asserts on the typed object.
- **Biomarker awareness:** prompt explicitly requires any low/high biomarker to surface as a clinician-discussion item.

Plan Writer (`plan_writer_agent`)

- **Role:** assemble the final memo.
 - **Hard rules in the prompt:** every recommendation references the profile (“Since you mentioned...”); every action has a number; if labs are provided, ≥ 2 Focus Areas must cite a biomarker by name + value.
 - **Required sections:** Your Profile · Focus Areas · Action Plan (Today / This Week / This Month) · Nutrition · Movement · Sleep & Recovery · What to Avoid · Safety Notes · When to See a Professional · If you only do one thing this week · disclaimer.
-

4. CAMEL primitives used

These are the **specific CAMEL classes** that show up in this project. Know them cold — these are the headline talking points.

Primitive	Where	What it does
ChatAgent	every agent	The cornerstone class. (system_message), memory (default), tools, response_format for output. .step(message) core call.
ModelFactory.create(...)	src/agents.py::_model, src/lab_parser.py	Builds a model backend. ModelPlatformType.0 ModelType.GPT_40 + model_config_dict w temperature, stream stream_options.incl
FunctionTool	src/agents.py	Wraps a Python callable the LLM can call. We use search_duckduckgo, search_health_kb, query_health_graph.
SearchToolkit	src/agents.py	CAMEL's built-in toolkit search_duckduckgo.
Workforce	src/workforce.py, backend/runner.py	Hierarchical orchestration. Coordinator + task-plan dispatch subtasks to workers based on each worker's description.
add_single_agent_worker(description, worker)	src/workforce.py, backend/runner.py::_build_instrumented_workforce	Registers a ChatAgent worker with a natural-language description used for routing.
Task	backend/runner.py	The unit of work passed to Workforce.process_task. Has content + id.
Workforce.set_stream_callback(cb)	backend/runner.py	Sub-agent token stream. Callback signature (worker_id, task_id, text, mode)
process_task_async(task)	backend/runner.py::_run	Async execution. Returns with .result = final answer text.
ChatAgent.on_request_usage	backend/runner.py::_usage_callback	Per-request token usage. We aggregate per-worker cost telemetry.
response_format=PydanticModel	src/lab_parser.py::parse_lab, evals/*	Structured output. The parser returns a typed object if possible, else free text.

Things to say about CAMEL specifically:

- “CAMEL is unusual because the framework ships with a clear research thesis — the scaling-law-of-agents idea from Guohao’s paper. Workforce is the productized layer of that thesis. Eigent’s product is the productized layer of Workforce.”

- “I deliberately used *Workforce* for dispatch, not custom orchestration. The coordinator decides how to decompose the root task and picks workers by description — that’s the auto-decompose mode. I also do manual orchestration for the follow-up flow (where I want deterministic 2-stage behavior), and I justify the split.”
-

5. Retrieval: vector + graph + web (agentic RAG)

The Researcher has three tools. The LLM **chooses** which to call per query based on prompt guidance. This is **agentic** retrieval (vs. always-on RAG where every query hits the vector store regardless).

Vector RAG — `src/rag.py`

- **Source corpus:** `data/kb_sources.txt` — ~30 curated URLs (NIH ODS fact sheets, CDC physical activity / sleep / nutrition, AHA, USDA Dietary Guidelines, NHLBI, Mayo Clinic).
- **Ingestion:** `scripts/ingest_kb.py` —
 1. **Firecrawl** (`firecrawl-py v4`) fetches each URL → clean markdown.
 2. Paragraph-greedy chunker (~500 tokens with single-paragraph overlap), measured via `tiktoken.cl100k_base`.
 3. **Local embeddings** via `sentence-transformers/all-MiniLM-L6-v2` (384-dim, runs on CPU, no API key, query text never leaves the box).
 4. **Idempotent upsert** to Qdrant using deterministic UUIDs derived from `sha1(source_url + chunk_index)`.
- **Storage:** Qdrant in a local Docker container (`docker-compose.yml`). Collection `health_kb` with cosine distance.
- **Query path:** `search_health_kb(query, k=5)` → embed → top-k cosine → return chunks with source URL + title + score.

Talk-track: “Embeddings run locally on the box — that’s the *Eigent* local-first posture. The vector store is Qdrant, the same thing you’d ship to production; the dev path is the local Docker container, the prod path is Qdrant Cloud with the same client code.”

Graph RAG — `src/graph_rag.py`

- **Source:** `data/health_graph.yaml` — hand-authored, ~65 entities (nutrients, conditions, biomarkers, foods, exercise) and ~124 typed edges. Edge predicates: `addresses`, `found_in`, `measured_by`, `interacts_with`, `risk_factor_for`, `contraindicated_with`.
- **Storage:** NetworkX `MultiDiGraph` loaded once at startup. In-memory, sub-millisecond traversal, ~10 KB. No extra service.
- **Query:**
 1. Embed query text with the **same** local sentence-transformers model (no second model = no extra memory footprint).
 2. Embed each entity’s name+aliases+description at startup; cache.
 3. Cosine top-k entities, then return each with its 1-hop neighborhood (both outgoing and incoming edges).
- **Why graph beats vector for some queries:** *relational* questions. “What foods contain magnesium?” → top entity = magnesium → traverse `found_in` edges. Vector RAG returns scattered passages.

Talk-track: “Vector RAG is for unstructured guideline questions; the graph is for relational questions where the answer is a set of typed relationships. The Researcher’s prompt steers per query — prefer the graph for ‘what addresses X?’, the KB for ‘how much X is recommended?’.”

Web search

- `SearchToolkit().search_duckduckgo` from CAMEL.
- Only fires when the curated sources are unlikely to have the answer (product names, recent news, niche conditions). In practice, almost never fires on a typical health profile.

How the runtime sees a retrieval

Every retrieval is wrapped (`backend/runner.py::_wrap*_tool`) so that the call emits a typed `tool_call` SSE event with the query and the retrieved sources/entities. The UI's `WorkerNode` then shows badges (🌐 N graph · 📖 N KB · 🌐 N web), and the drawer lists each retrieval inline with similarity scores + source URLs.

6. Lab Parser — typed document ingestion

- **Endpoint:** `POST /api/labs` (multipart) — accepts a PDF file or pasted text. PDF text is extracted via `pypdf`.
- **Agent:** `lab_parser_agent()` in `src/lab_parser.py` — a standalone `ChatAgent` with `response_format=BiomarkerPanel`.
- **Schema:** `src/schema.py::Biomarker` + `BiomarkerPanel`. Each biomarker has `name`, `value` (string — handles numeric + qualitative), `unit`, `reference_range`, `flag` ∈ {normal, low, high, unknown}.
- **Visualized as the 5th node:** `frontend/src/components/ParserNode.tsx` shows the parser as a 5th node *above* the Coordinator with status driven by `labLoading` + `labPanel` presence.
- **Threading into the Workforce:** the parsed biomarkers are sent in the `POST /api/run` body; `backend/runner.py::_format_biomarkers` renders them as a labeled bullet block inside the root-task `content`. Every downstream worker sees them.
- **Prompt updates ensure use:** the Safety Reviewer must flag any low/high biomarker as a consult item; the Plan Writer must reference at least two biomarkers by name + value in Focus Areas.

Talk-track: “This is the product moment. The user uploads their blood work, the Lab Parser extracts a typed `BiomarkerPanel`, and the downstream Workforce members ground every recommendation in the person’s actual numbers. The Safety Reviewer treats out-of-range markers as consult items; the Plan Writer cites them by name.”

7. Human-in-the-loop approval gate

The HITL gate sits **after** the Workforce finishes but **before** the plan is released to the user. The run pauses; the user reviews a preview; only on Approve does the full plan render.

Backend — backend/runner.py

```
# Pseudocode of the gate
result = await wf.process_task_async(task)
memo = _extract_memo(result.result) # also strips reasoning prelude

approval_future = loop.create_future()
_pending_approvals[task_id] = approval_future
emit(RunEvent(type="human_input_required", memo=memo)) # ← UI shows modal

try:
    approved = await asyncio.wait_for(approval_future, timeout=300)
except asyncio.TimeoutError:
    emit(RunEvent(type="error", text="Approval timed out..."))
else:
    if approved:
        emit(RunEvent(type="task_complete", memo=memo))
        _finished_runs[task_id] = FinishedRun(profile, biomarkers, memo)
    else:
        emit(RunEvent(type="error", text="Run rejected by user. "))
finally:
    _pending_approvals.pop(task_id, None)
```

- **Endpoint that resolves the gate:** POST /api/run/{task_id}/human_input → calls `resolve_approval(task_id, approved) → Future.set_result(...)`.
- **Timeout (5 min):** prevents zombie tasks if the user disappears.

Frontend — frontend/src/components/ApprovalModal.tsx

The modal parses the memo's # H1 sections (Your Profile, Safety Notes, Focus Areas, If you only do one thing this week) and shows a preview. The full memo only renders **after** approve. The total cost ticker stays visible — the user knows what they spent before approving.

Why this maps to Eigent

Eigent's marketing leads with HITL: their desktop app gates high-stakes agent actions (code exec, file writes, tool installs) behind explicit user consent. This pattern — *pause on an `asyncio.Future`, expose a typed event, resolve from a POST* — is the production-grade version of what you'd see in a real Eigent feature.

8. Follow-up refinement (the 2-stage micro-run)

The big “product-engineering” feature. After a plan is approved, the user adds new context (“actually my left knee hurts”). The system runs **only the Safety Reviewer + Plan Writer** against the prior memo + the new note — not the full pipeline.

Why bypass the Workforce

The Workforce coordinator would re-decompose the task and possibly re-fire Researcher and Assessor — wasteful (their work is already in the memo) and slow. A deterministic 2-stage flow is the right shape for refinement.

How it's wired — backend/runner.py

- **State:** an in-memory `_finished_runs: dict[task_id, FinishedRun]` is populated on approval. `FinishedRun` holds `profile, biomarkers, memo`.

- `start_follow_up(orig_task_id, note)` : creates a new `task_id` (`<orig>-fXXXX`) and schedules `_run_followup`.
- `_run_followup` :
 1. Build Safety Reviewer + Plan Writer agents manually (NOT via Workforce), each with its own `on_request_usage` callback for cost tracking.
 2. Emit `task_started` + `worker_running(role=critic)`.
 3. `safety_agent.step(prev_profile + prev_memo + new_note)` — full response captured via `asyncio.to_thread` to avoid blocking the event loop.
 4. Emit the safety result as a single `worker_chunk` with `mode="accumulate"` (no token streaming for follow-ups — keeps the code simple).
 5. `worker_running(role=summarizer)` → Plan Writer revises.
 6. Strip reasoning prelude → emit `human_input_required` → wait on Future → on approve, emit `task_complete` and write the updated `FinishedRun` back so **chained follow-ups** work.

Frontend — `frontend/src/components/MemoPanel.tsx` + `App.tsx`

- The MemoPanel renders a follow-up textarea + button **only when** `phase === "done"`.
- `App.runFollowUp(note)` :
 1. `store.startFollowUp()` — resets ONLY Safety + Plan Writer state (keeps Researcher/Assessor as done — they're carried over).
 2. POST `/api/run/{taskId}/follow_up`.
 3. Store the new `task_id`, open a fresh SSE.
- The graph visualizes continuity: the two relevant nodes re-light, the other two remain done.

Cost story

- A full run: ~\$0.025–0.04 (4 workers + coordinator).
- A follow-up: ~\$0.005–0.01 (2 agents, prior memo as context).

Talk-track: “I bypass the Workforce coordinator for follow-ups because the prior agents’ work is already encoded in the memo. Running them again would be wasteful. The two relevant workers run manually with `asyncio.to_thread` so the event loop stays free. State is held in an in-memory dict — fine for a single-instance demo, with a clear migration path to Redis or a real DB in production.”

9. Chain-of-thought transparency

The user audit trail. Each agent emits two top-level `##` H2 sections:

```
## Reasoning
- 3–5 bullets covering what was considered, ruled out, and chose because...

## Conclusion
[the normal output]
```

Backend — `backend/runner.py::_strip_reasoning_prelude`

The final user-visible memo strips everything up through the `## Conclusion` line. The reasoning is preserved in the per-worker streamed output for the drawer.

Frontend — `frontend/src/components/WorkerDrawer.tsx::splitReasoning`

Parses the streamed text into `(reasoning, rest)` by finding the `## Reasoning` and `## Conclusion` markers (case-insensitive). The drawer renders the reasoning trace in a distinct violet block above the Conclusion. The `WorkerNode` card gets a small  `reasoning` badge.

Why this matters for healthcare

“In a health tool, the audit trail matters as much as the answer. The Safety Reviewer saying ‘follow-with-caution’ isn’t useful unless I can see WHY. I made every agent emit explicit reasoning so the user can inspect it without re-running the system.”

CoT transparency vs. CoT data generation (be ready for this)

- **CoT transparency** (this project): reasoning visible at inference time for trust + auditability.
 - **CoT data generation** (CAMEL has `CoTDataGenerator`, not used here): synthesizes `(problem, reasoning, answer)` triples to *train* models on. Different goal entirely — not relevant when consuming a frontier model. **Talking point only.**
-

10. Live cost telemetry

The streaming-usage gotcha

OpenAI’s streaming responses **don’t include token usage by default**. You have to opt in via:

```
model_config_dict={"temperature": 0.2, "stream": True,
                  "stream_options": {"include_usage": True}}
```

Without that, the `on_request_usage` callback fires with zeros — and your “live cost” stays at \$0. This is in `src/agents.py::_model`.

Per-worker tracking

Each agent built in `backend/runner.py::_build_instrumented_workforce` gets a pinned `on_request_usage` callback that accumulates `prompt_tokens` + `completion_tokens` per worker. The callback emits a `worker_usage` SSE event with cumulative tokens and computed cost (using GPT-4o pricing constants in the same file).

UI

- `frontend/src/components/WorkerNode.tsx` shows `tokens · $0.XXXX` in each card’s footer.
- `frontend/src/App.tsx` shows a header cost ticker via `selectTotalCost` (sums per-worker costs).
- After a run, the user sees a real per-stage breakdown — ~\$0.013 for the Researcher (web/KB-heavy), ~\$0.005 for the Assessor, etc.

Talking points

- “Cost telemetry is a product feature — it builds trust and lets the user catch runaway agent behavior. *Eigent’s roadmap literally lists cost tracking and prompt caching as priorities.*”
 - “Prompt caching would be the obvious next optimization — the long static system prompts are exactly what `cache_control` breakpoints are for, and OpenAI’s `prompt_cache_hit_tokens` would flow into the same usage hook. ~80–90% reduction on cached input.”
-

11. Product engineering stack

The full-stack surface mirrors Eigent's actual stack. Be ready to name every piece.

Backend

- **FastAPI** (`backend/server.py`) — async-native, Pydantic-typed routes, free OpenAPI docs at `/docs` .
- **Uvicorn** — ASGI server with `--reload` in dev.
- **sse-starlette** — SSE wrapper that handles keep-alive comments, reconnection on the browser side via `EventSource` .
- **pydantic v2** — every event is a typed `RunEvent` ; same for request/response bodies.
- **python-dotenv** — `.env` loading for local dev (`load_dotenv()` on startup).
- **uv** — Astral's Python package manager. `uv sync` reproducibly installs from `uv.lock` . ~10× faster than `pip`.

Frontend

- **Vite + React + TypeScript** — fast HMR, esbuild transforms.
- **Zustand** — state container. Per-domain state (workers, lab panel, phase, task_id, expanded role) plus `applyEvent` reducer that handles every SSE event type.
- **React Flow** (`reactflow v11`) — the live agent graph. Custom node types for `worker` and `parser` . Animated edges when a worker is running. `onNodeClick` for the click-to-expand drawer.
- **Tailwind CSS** — utility-first styling. Custom CSS for the markdown memo (no typography plugin to keep deps lean).
- **react-markdown** — renders the final memo + sections inside the approval modal.

Deployment

- **Multi-stage Dockerfile** — Node stage builds the React app; Python stage installs `uv` + deps, copies the built `frontend/dist/` , runs Uvicorn. Single deployable container.
- **render.yaml** — Render Blueprint config. Env vars `OPENAI_API_KEY` + `APP_PASSWORD` . (Note: the deployed image predates RAG/labs/HITL — those run locally via Docker Compose for the Qdrant dependency.)

Why this is “Eigent-shaped”

Per Eigent's README: their backend is FastAPI + uv + Uvicorn, their frontend is React + TS + Zustand + React Flow + Tailwind, and their product surfaces MCP integrations + HITL checkpoints. The parallels are intentional.

12. Evals

Three scripts in `evals/` :

`deterministic.py` — pressure-test the Safety Reviewer

Calls `safety_reviewer_agent()` directly with `response_format=SafetyReview` and a clearly-risky profile (T1D + 5-day water fast + stop insulin + 10 km daily runs). Asserts:

- `len(risks) >= 2`
- `len(consult_a_professional) >= 1`
- `verdict != "safe-to-follow"`

Why deterministic-style: the assertion is *structural*, not LLM-as-judge. The Safety Reviewer either surfaces real risks on a risky input or fails the test. It catches regressions cheaply.

`llm_judge.py` — rate generated plans

Runs the full Workforce on three sample profiles (healthy college student, sedentary middle-aged, T2 diabetic on metformin), then a **separate judge agent** scores each plan 1–5 on:

- **coherence** — internally consistent, clear
- **actionability** — concrete things to do this week
- **safety** — appropriate caution + escalation
- **personalization** — fit to this specific profile

Scores append to `evals/results.csv`.

`cost_table.py` — per-worker token + cost + wall time

Builds agents with `on_request_usage` callbacks (same pattern as the runner), runs one full pipeline, prints a markdown table for the README.

Talk-track: “My three evals layer cleanly: deterministic for structure, LLM-judge for quality, cost-table for the production-relevant metrics. The deterministic eval catches regressions; the judge catches quality drift; the cost table is the data behind decisions like ‘is prompt caching worth it?’”

13. Files map (where everything lives)

```
eigent-health-team/
├── pyproject.toml          # uv-managed deps
├── docker-compose.yml     # Qdrant local container
├── Dockerfile             # multi-stage Node + Python build
├── render.yaml            # Render single-service config
├── .env.example           # OPENAI + FIRECRAWL + APP_PASSWORD
├── src/                   # Core CAMEL logic – powers CLI + web app
│   ├── schema.py          # Pydantic models: SafetyReview, Biomarker, BiomarkerPanel
│   ├── agents.py          # 4 Workforce ChatAgent builders + system prompts
│   ├── lab_parser.py      # 5th agent: LabParser (typed BiomarkerPanel via response_format)
│   ├── workforce.py       # build_workforce() – the CLI wires
│   ├── rag.py             # Qdrant + sentence-transformers retriever
│   ├── graph_rag.py       # NetworkX MultiDiGraph + embedded query
│   └── main.py            # CLI entry: profile → memo
├── backend/               # FastAPI app
│   ├── events.py          # Typed RunEvent
│   ├── runner.py          # Async runner: Workforce + tool wraps + HITL + follow-ups
│   └── server.py          # Routes: /api/run, /api/labs, /api/run/{id}/human_input,
│                           # /api/run/{id}/follow_up, /api/prompts, /api/health
├── frontend/              # React + TS + Vite
│   └── src/
│       ├── store.ts       # Zustand store + applyEvent reducer
│       ├── App.tsx        # Top-level: lab upload, run + follow-up handlers
│       ├── lib/sse.ts     # EventSource wrapper
│       └── components/
│           ├── Gate.tsx   # Password gate
│           ├── BiomarkerTable.tsx
│           ├── ParserNode.tsx # Lab Parser node (React Flow)
│           ├── WorkerNode.tsx # 4 worker nodes (status, tokens, cost, badges)
│           ├── TaskGraph.tsx # React Flow canvas wiring
│           ├── WorkerDrawer.tsx # Click-to-expand: prompt, reasoning, tool calls, output
│           ├── MemoPanel.tsx # Markdown plan + follow-up input
│           └── ApprovalModal.tsx # HITL gate UI
├── data/
│   ├── kb_sources.txt     # ~30 URLs for the Qdrant KB
│   └── health_graph.yaml  # 65 entities, 124 edges
├── scripts/
│   └── ingest_kb.py       # Firecrawl → chunk → embed → Qdrant upsert
├── evals/
│   ├── deterministic.py   # SafetyReview structural assertion
│   ├── llm_judge.py       # 3-profile 4-axis rating
│   ├── cost_table.py      # Per-worker token/cost/latency
│   └── results.csv        # llm_judge log
└── outputs/               # Sample generated memos (CLI)
```

14. Likely interview questions, with scripts

“Walk me through what you built.”

“It’s a 5-agent personalized health system built on CAMEL’s Workforce framework — the same framework Eigent ships on. A Lab Parser extracts a typed BiomarkerPanel from a PDF or pasted text, then a Workforce of four ChatAgents runs: Researcher, Health Assessor, Safety Reviewer, Plan Writer. The

Researcher has three retrieval tools and chooses per query — a knowledge graph of nutrient relationships, a vector store over curated guidelines, and the open web. The Workforce pauses on an `asyncio.Future` for a human-in-the-loop approval gate before the plan is released. After approval, the user can drop a follow-up — ‘actually my knee hurts’ — and only the Safety Reviewer and Plan Writer re-run, at ~1/10th the cost. Live token + cost telemetry per worker, click-to-expand drawer showing each agent’s reasoning trace and tool-call queries with retrieved sources or entities.”

“Why CAMEL and not LangGraph / AutoGen / CrewAI?”

“CAMEL has a research thesis baked in — Guohao’s scaling-law-of- agents paper. Workforce is the productized layer of that thesis, and Eigent is the productized layer of Workforce. Building on CAMEL puts me in the same conceptual model as the team. LangGraph is graph-state runtime — great if your workflow is a known DAG; not what I needed. CrewAI is more opinionated/role-based with less research depth. AutoGen is similar shape but heavier MS tooling.”

“How does the Workforce coordinate work?”

“By default, the Workforce has a coordinator agent and a task- planner agent that CAMEL provides. When I pass a root Task, the planner decomposes it into subtasks, and the coordinator picks the right worker for each based on the natural-language `description` each worker is registered with. That’s why my worker descriptions are specific — ‘Health Researcher — gathers evidence-based health information using web search; use for any subtask that needs facts from the web.’ One thing I observed: the coordinator is non- deterministic; sometimes it folds a stage. For follow-ups where I wanted strict 2-stage behavior, I bypassed the coordinator and called `ChatAgent.step()` manually.”

“How does the agent decide which retrieval tool to use?”

“Pure prompt steering. The Researcher’s system prompt declares three tools — `query_health_graph` , `search_health_kb` , `search_duckduckgo` — and tells the model when to prefer each: graph for relational questions, KB for guidelines, web for fresh/ specific things. The LLM emits a tool call; CAMEL’s `FunctionTool` machinery dispatches it; the result feeds back as an observation on the next loop. That’s the ‘agentic’ part — the LLM, not the runtime, makes the routing decision.”

“Vector RAG vs Graph RAG — why both?”

“They answer different question types. Vector RAG is good for guidelines — ‘how much vitamin D is recommended for adults’ — the answer is a paragraph in the KB and similarity gets you there. Graph RAG is good for relationships — ‘what foods contain magnesium, what biomarkers measure iron status’ — vector retrieval returns scattered text; the graph traverses typed edges and returns structure. I curated the graph by hand to start, ~65 nutrient-centric entities. At scale, an agent could build the graph from the KB chunks — extract entities and relations, write edges — but for a demo, hand-curated is faster and ground-truth.”

“Why local embeddings instead of OpenAI’s?”

“Two reasons. First, it’s free and fast — sentence-transformers `all-MiniLM-L6-v2` is 384-dim, runs on CPU, no API key, and the embed step doesn’t add latency to my run path. Second, it’s the Eigent local-first posture — user query text never leaves the machine for retrieval. Inference still goes to OpenAI in my

current build; swapping to Ollama would be a one-line model change to make the whole thing run locally.”

“Tell me about your HITL approval gate.”

“After the Workforce produces the memo, I pause the run on an `asyncio.Future` registered per-task. I emit a typed `human_input_required` SSE event with the memo attached, and the UI opens an approval modal showing the safety verdict, the focus areas, and the ‘if you only do one thing’ line — but the full plan stays hidden until approve. The user clicks Approve, which POSTs to `/api/run/{id}/human_input`; that resolves the Future; the runner emits `task_complete` and the plan renders. There’s a 5-minute timeout that cancels the run gracefully if the user walks away. The Approve POST is rate-limit-aware and password-gated, same as the run POST.”

“How does the follow-up flow work cost-wise?”

“Instead of re-running the four-agent Workforce, follow-ups manually run only the Safety Reviewer and Plan Writer against the stored prior memo plus the new note. That’s two model calls instead of six-plus, no web/KB/graph retrieval, no coordinator decomposition. About \$0.005–0.01 per follow-up vs. \$0.025–0.04 for a fresh run. State is kept in an in-memory dict `_finished_runs[task_id]`; chained follow-ups work because the approved follow-up writes its own `FinishedRun` back.”

“What evals did you build?”

“Three. A **deterministic** structural assertion — Safety Reviewer must return ≥ 2 risks and a non-safe verdict on an obviously-risky profile (T1D + 5-day fast). An **LLM-judge** that rates 3 generated plans on four axes — coherence, actionability, safety, personalization. And a **cost table** that instruments a run with `on_request_usage` callbacks and prints per-worker tokens + cost + wall time. The three layer cleanly: structural for regressions, judge for quality, cost for production-relevant metrics.”

“What was the hardest part?”

Pick one of these and tell it as a story:

- **The streaming-usage gotcha.** Live cost stayed at \$0 until I realized OpenAI’s streaming responses omit usage by default — needed `stream_options.include_usage=true` in the model config.
- **Workforce auto-decompose non-determinism.** The coordinator occasionally folded one of the four stages. I considered Pipeline mode but bounded the scope and accepted the dynamic behavior for the main flow; for follow-ups where I needed determinism, I bypassed Workforce and ran agents manually.
- **PDF / Firecrawl API version drift.** First ingestion run failed because the new firecrawl-py uses `Firecrawl(api_key).scrape(url, formats=[...])`, not the older `FirecrawlApp.scrape_url(params=...)`. Quick fix, but a reminder to pin or guard against vendor API churn.

15. What I’d build next (and why I didn’t)

Have these ready — they show you know what’s beyond the demo.

- **Prompt caching** with Anthropic `cache_control` breakpoints or OpenAI’s prefix caching. Big static system prompts → ~80–90% cheaper after the first call. Telemetry already in place; would just add `cache_read_input_tokens` to the rollup.

- **Local model option (Ollama).** Swap `ModelPlatformType.OPENAI` for an OpenAI-compatible local URL. Slower but \$0/run; the Eigent local-first story end-to-end.
 - **Persistent memory** via CAMEL's `LongtermAgentMemory` + SQLite. *“Last time you said you started walking 20 min/day — how's that going?”* Multi-session product behavior.
 - **MCP integration.** Expose `search_health_kb` and `query_health_graph` as MCP servers; let any MCP-compatible client (including Eigent) consume them.
 - **Graph RAG with a mini React Flow viz in the drawer.** The retrieved subgraph rendered as nodes + edges — bigger demo-wow but ~2h of pure UI work.
 - **Auto-graph build from KB chunks.** A graph-building agent that reads the vector chunks and writes the YAML. Scales the graph beyond what I'd hand-curate.
 - **CoT data generation** — CAMEL has `CoTDataGenerator`. Useful if fine-tuning a smaller model on this domain.
-

16. Honest limitations

When pressed, own these:

1. **Workforce coordinator non-determinism.** Sometimes folds a stage. Acceptable for a demo with prompt steering; a production build would either use Workforce Pipeline mode or do manual orchestration like the follow-up flow.
 2. **PDF parsing is text-only.** Scanned/image PDFs need OCR. Out of scope; documented with a “paste text” fallback in the UI.
 3. **In-memory follow-up state.** `_finished_runs` is a dict; a server restart loses it. Migration path: Redis or SQLite.
 4. **Vector KB is small (~30 sources, 179 chunks).** Real production would have orders of magnitude more, with periodic re-ingestion.
 5. **Graph is hand-curated (~65 entities).** Won't scale to the long tail without an extraction pipeline.
 6. **Single-user.** No accounts, no tenancy. APP_PASSWORD is the only gate. Enterprise multi-tenancy is a known follow-on.
 7. **No prompt caching yet.** Easy win, deferred for time.
 8. **Free-tier Render cold starts.** ~50s + heavy camel-ai imports. Demo runs locally; deployment is for show-and-tell.
 9. **DDG free endpoint is sparse.** Tavily or Exa would harden the web search; not worth the API integration time for the demo.
-

Final check before walking in

- Memo the elevator pitch and the 4-min demo script.
- Run the demo end-to-end yourself once. Time it.
- Read §3 (Agents) and §4 (CAMEL primitives) out loud.
- Practice answering “walk me through what happens when a user clicks Run” without notes.
- Know the 5 honest limitations in §16 — they're conversation starters, not weaknesses.

Educational information only. Built on [CAMEL-AI](#) for interview prep at [Eigent](#).